

that each rupee spent on infrastructure will increase production by Rs 2. So, you can formulate the problem as:

```
c - p = 0 # consumption equals production
c - r = 0 # consumption equals expenditure on rural employment
p - 2*i = 0 # production equals twice the expenditure on infrastructure
r + i = 100 # sum of the expenditures must be 100
```

You can use SciPy's `linalg` module to solve this problem as follows, where the four columns are c, p, r, and i:

```
>>> from scipy import linalg
>>> A = [[1,-1,0,0],[1,0,-1,0],[0,1,0,-2],[0,0,1,1]]
>>> b = [0,0,0,100]
>>> linalg.solve(A,b)
array([ 6.66666667, 6.66666667, 6.66666667, 33.33333333])
```

Let's assume that the government can also waste money, w. That is:

```
r + i + w = 100
```

You now have 4 equations but 5 unknowns. So, you can use the `lstsq` method and get a solution:

```
>>> A = [[1,-1,0,0,0],[1,0,-1,0,0],[0,1,0,-2,0],[0,0,1,1,1]]
>>> linalg.lstsq(A,b)
(array([[ 27.27272727],
        [ 27.27272727],
        [ 13.63636364],
        [ 99.09090909]]), array([], dtype=float64), 4, array([ 2.53498934,
1.87927105, 1.13663538, 0.86628904]))
```

The first array in the result contains the recommended values for the five parameters. The solution is undesirable, even if realistic. It suggests that 59 per cent of the money be wasted! Your model will need to minimise wastage.

Optimising in the face of constraints

SciPy, as yet, does not include a module to optimise a utility function subject to constraints. One project extending SciPy for this functionality is `OpenOpt`. `OpenOpt` uses *python-cvxopt* as one of the solvers. So, you will need to install *python-cvxopt*, which is available in the Fedora or Ubuntu repository. `OpenOpt` can be installed from openopt.org.

Define the optimisation problem using an *lp* example described on the `OpenOpt` site, as follows:

```
import scipy as np
from openopt import LP
# Parameters (c, p, r, i, w)
# Minimize w
f = np.array([0,0,0,0,1])
# Equality constraints
# Same 4 eq as above
```

```
Aeq = np.array([[1,-1,0,0,0],[1,0,-1,0,0],
               [0,1,0,-2,0],[0,0,1,1,1]])
beq = (0,0,0,100)
# lower bounds - all values must be >= 0
lb = [0,0,0,0,0]
# Define and solve the problem
p = LP(f, Aeq=Aeq, beq=beq, lb=lb)
r = psolve('cvxopt_lp')
p.debug = 1
# The optimum values
print 'x_opt:', r.xf
```

If everything goes well, you should see a result that shows wastage as zero:

```
solver: cvxopt_lp problem: unnamed goal: minimum
pcost dcost gap pres dres k/t
0: 5.9091e+01 5.9091e+01 2e+02 4e-16 3e+00 1e+00
1: 3.7282e+01 3.7493e+01 3e+01 3e-14 4e-01 3e-01
2: 4.7204e-01 5.4936e-01 1e+00 1e-14 1e-02 8e-02
3: 4.7330e-03 5.5094e-03 1e-02 2e-14 1e-04 8e-04
4: 4.7331e-05 5.5096e-05 1e-04 3e-16 1e-06 8e-06
5: 4.7331e-07 5.5096e-07 1e-06 3e-14 1e-08 8e-08
6: 4.7331e-09 5.5096e-09 1e-08 3e-14 1e-10 8e-10
Optimal solution found.
lstsq: 1000 (optimal)
Solver: Time Elapsed = 0.1 CPU Time Elapsed = 0.01
objFunValue: 4.7331067e-09 (feasible, max constraint = 2.84217e-14)
x_opt: [ 6.66666667e+01 6.66666667e+01 6.66666667e+01
3.33333333e+01
4.73310667e-09]
```

You can explore alternate optimisation methods—for example, maximise c, (minimise -c) or minimise (w - c), etc. Your constraints can also include inequalities well.

`OpenOpt` includes non-linear optimisation tools as well. `SciPy` and extensions make it easy to formulate and explore solutions of the problems. `Matplotlib` makes it easy to visualise the results. So, you can focus on the creative and difficult part of defining the models and prevent any future crises in economics! 

References

- You can learn a lot from the courses available online. For example,
- Principles of Microeconomics: <http://ocw.mit.edu/OcwWeb/Economics/14-01Fall-2007/CourseHome/index.html>
 - Financial Markets: <http://oyc.yale.edu/economics/financial-markets/>
- The guest lecture on the management of the financial portfolio of Yale University is very interesting.

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com. Earlier articles in this series are available on <http://www.sethanil.com/python-for-friends>.